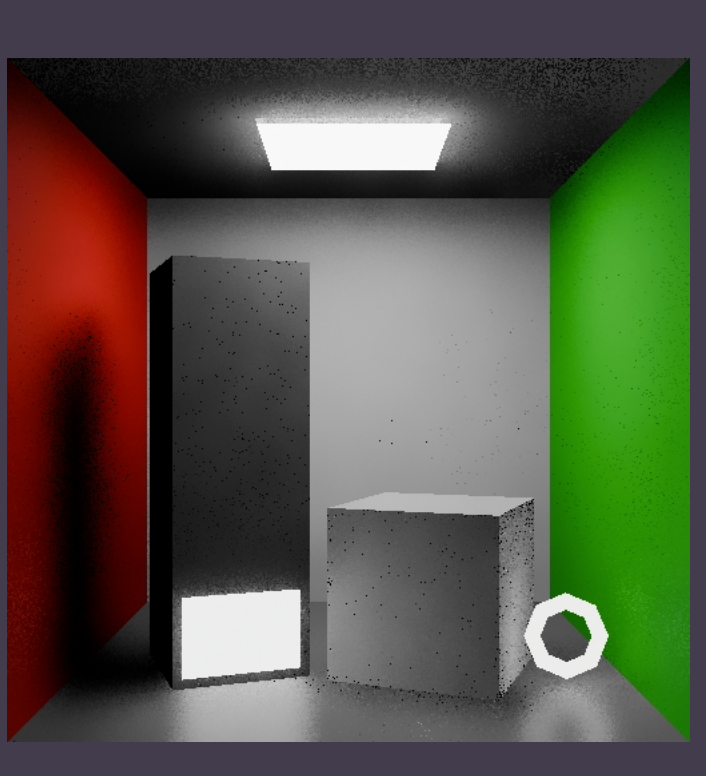
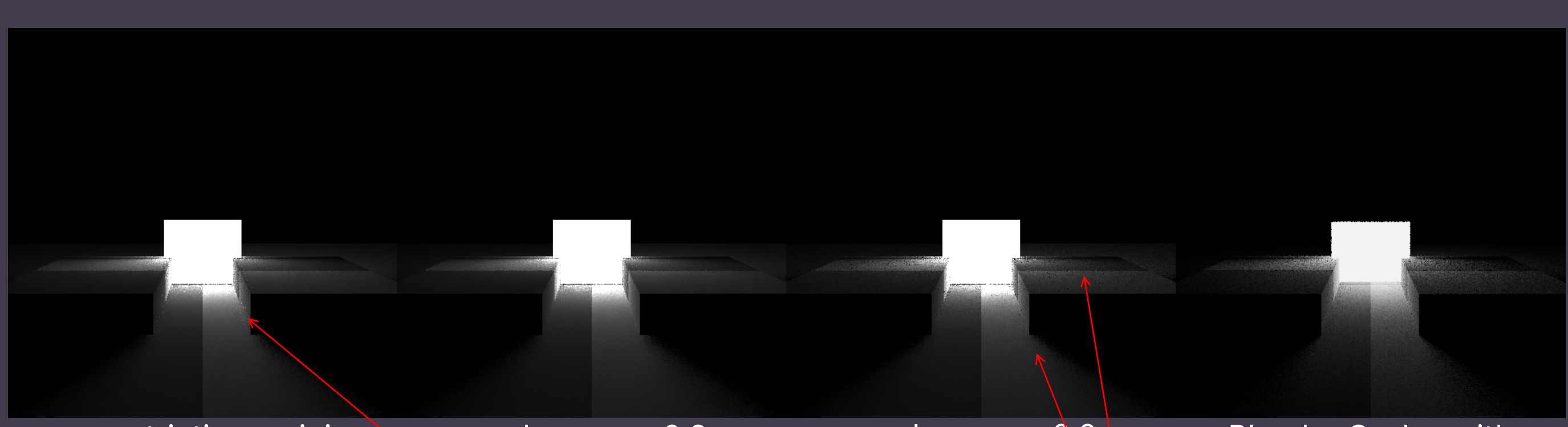
ReSTIR DI



ReSTIR 1 ray per pixel
BRDF: Lambert + GGX (different roughness)
Init samples 5 x MIS(Area Light Importance, GGX)
Temporal accumulation: 4 frames
Spatial accumulation: 12 samples

ReSTIR Lambert spatial join study in low res i.e. 512×512

BRDF: Lambert
Init samples 5 x Area Light Importance
Temporal accumulation: 2 frames
Spatial accumulation: 5 samples radius 10



no restriction on join

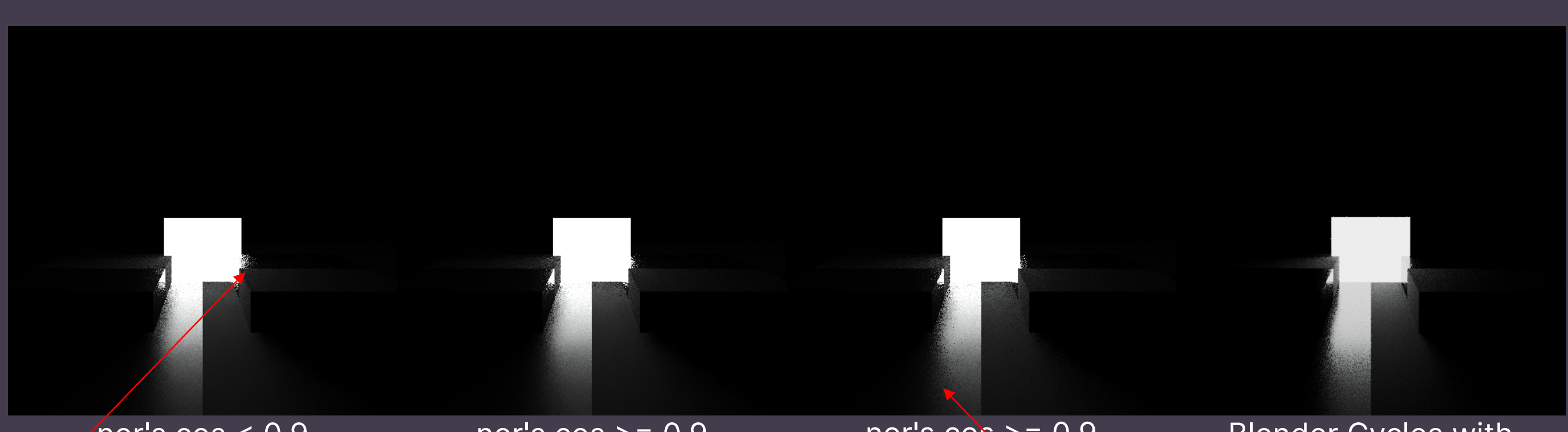
nor's cos >= 0.9
fireflies on box edge removed

nor's cos >= 0.9
pos's dist < 0.04
better shadow
lower convergence

Blender Cycles with 10 rays per pixel, no denoise, no indirect

ReSTIR GGX spatial join study

BRDF: GGX (different roughness)
Init samples 5 x MIS(Area Light Importance, GGX)
Temporal accumulation: 2 frames
Spatial accumulation: 5 samples radius 10



nor's cos < 0.9
pos's dist < 0.04
Specular over spreads

nor's cos >= 0.9
pos's dist < 0.04
half's cos >= 0.9
Adding restriction between half vectors narrows the specular

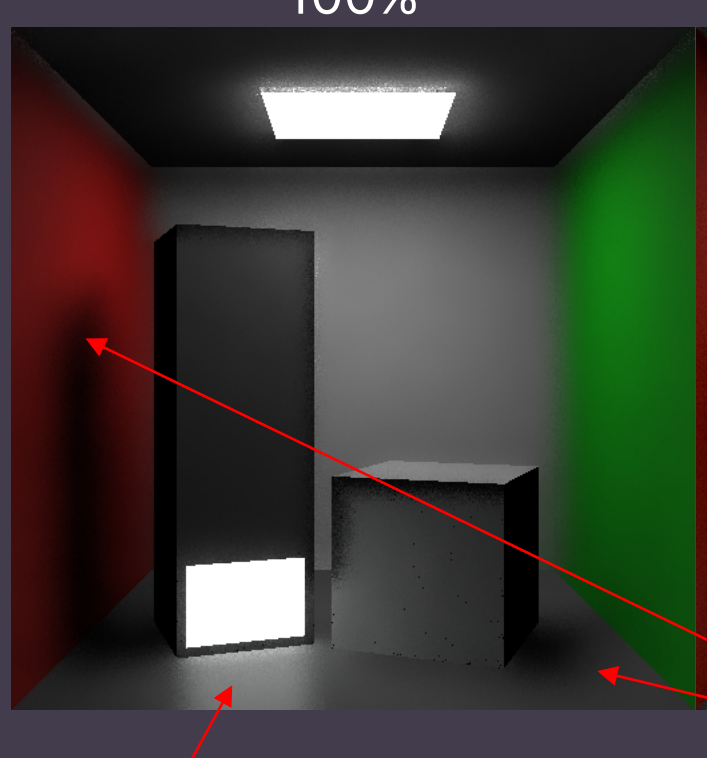
nor's cos >= 0.9
pos's dist < 0.04
half's cos >= 0.98
better specular
lower convergence

Blender Cycles with 10 rays per pixel, no denoise, no indirect

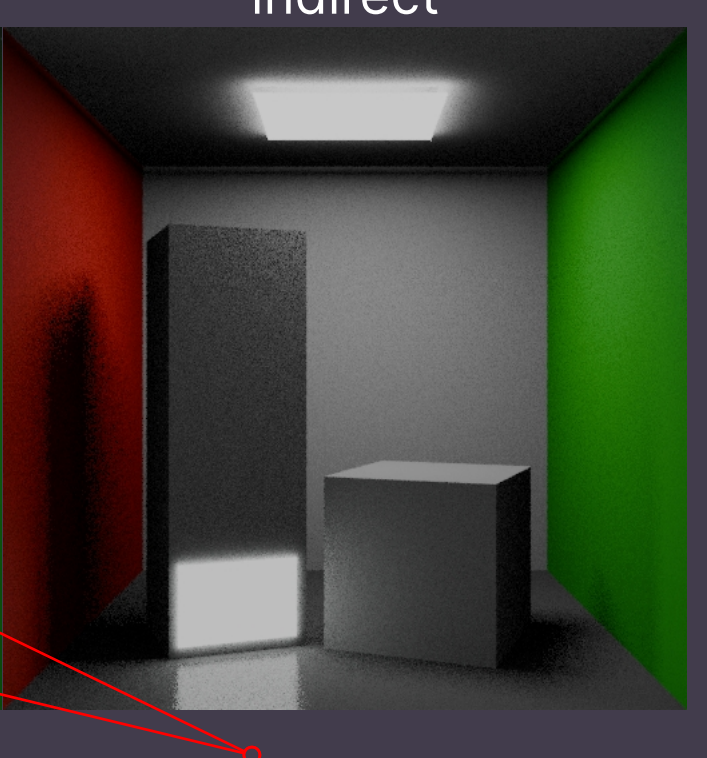
Average numbers of samples merged per pixel using different join restrictions:
73.51 [100%] - no restriction on spatial join
57.97 [78%] - pos's dist < 0.1
35.07 [47%] - pos's dist < 0.04 [good shadows]
33.12 [45%] - pos's dist < 0.1 and nor's cos >= 0.9 [OK shadows, good lambert]
26.35 [35%] - pos's dist < 0.1 and nor's cos >= 0.9 and half's cos >= 0.98 [ok shadows, good lambert, good ggx]
49.41 [67%] - half's cos >= 0.98

Lambert + GGX, 5 x MIS(Area Lights Importance, GGX), Temporal 4, Spatial 12 from radius 10 Visual impact & reservoir join percentage vs join restriction

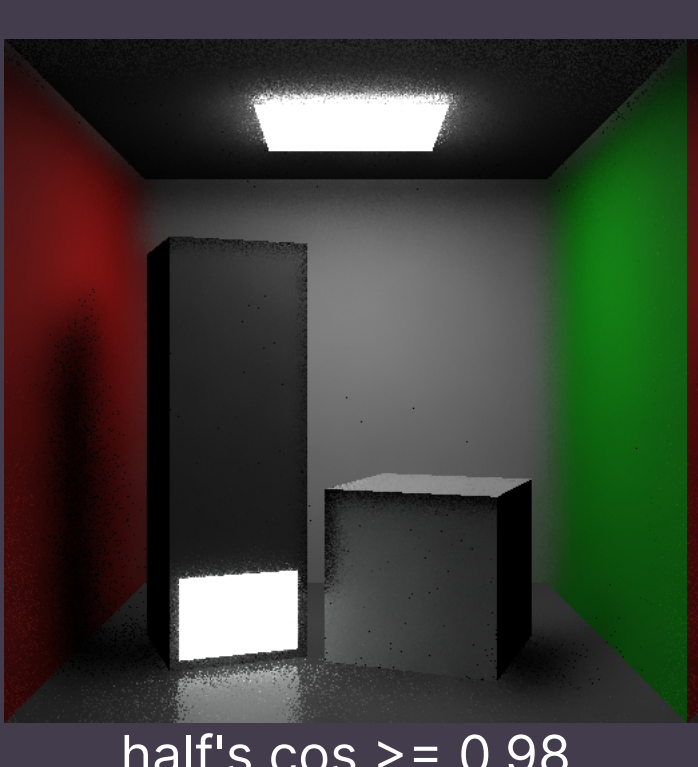
Start point
No restriction on spatial.
Average join: 13583.87
100%



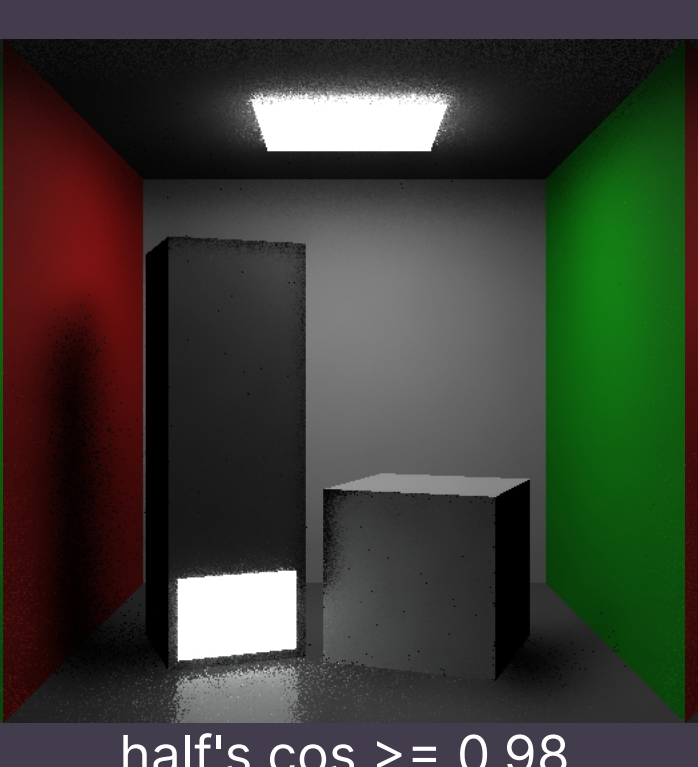
Visual reference
Blender Cycles with 10 samples, no denoise, no indirect



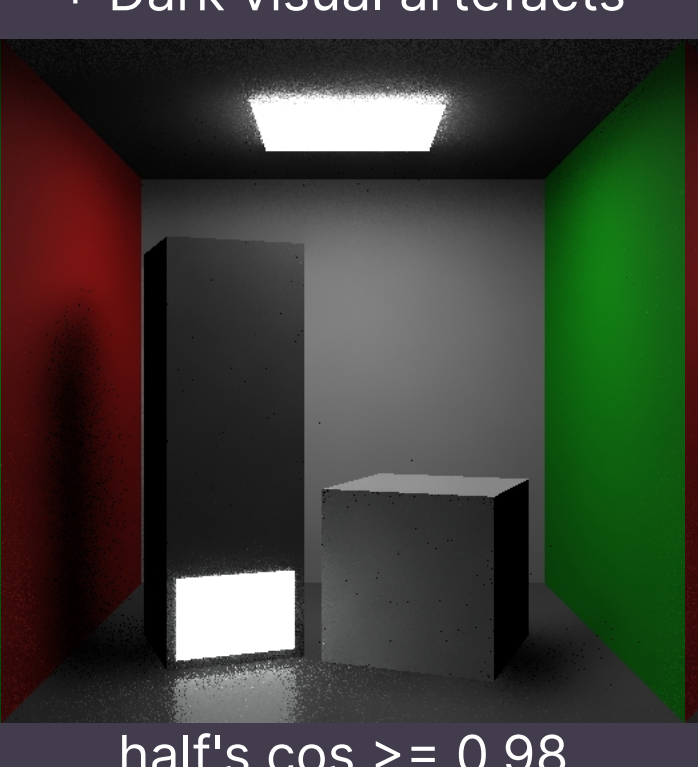
Washout specular + Rounded shadows
+ Dark visual artefacts



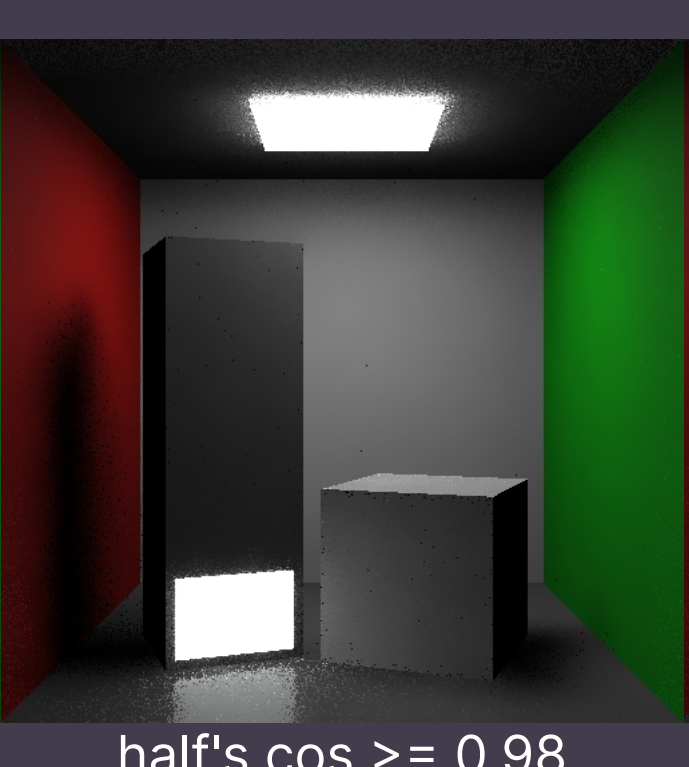
half's cos >= 0.98
average join: 6768.61
49.82%
+ Good specular



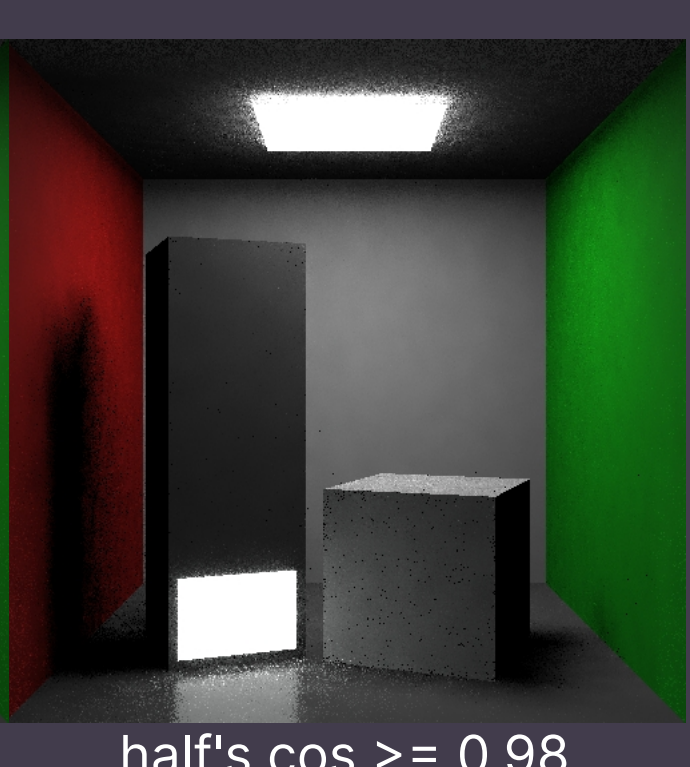
half's cos >= 0.98
nor's cos > 0.9
average join: 6478.03
47.68%



half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.5
average join: 6469.80
47.62%
+ No artefacts



half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.1
average join: 3692.08
27.17%
+ OK, shadows



half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.04
average join: 190.19
1.40%
+ Good Shadows

- Artefacts: nor's cos >= 0.9 and pos's dist < 0.5 is needed to remove artefacts

- Lambert: it looks good everywhere (once the artefacts are removed)

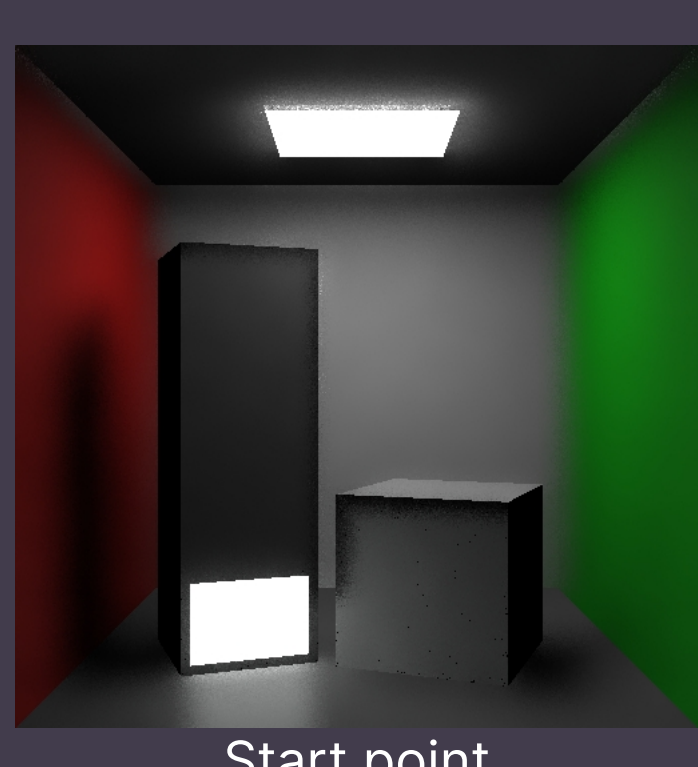
- GGX: half's cos >= 0.98 results in good specular but we drop half the spatial samples

- Shadows: best shadows with pos's dist < 0.04 (worse joining 1.4%). Acceptable shadows begin from pos's dist < 0.1, 27.17%

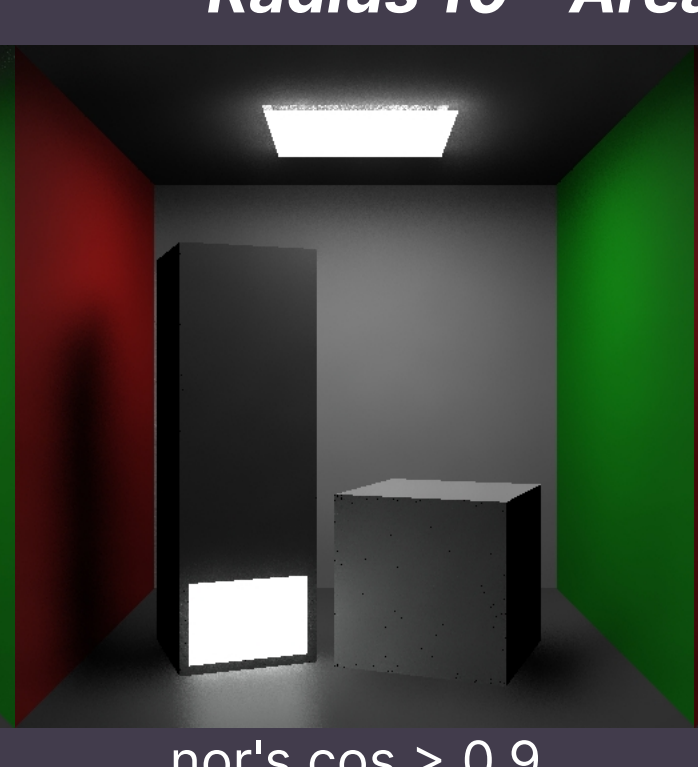
A lower percentage of considered neighbors joining means that convergence will require more frames.

Lambert + GGX, 5 x MIS(Area Lights Importance, GGX), Temporal 4, Spatial 12 samples no half's restriction

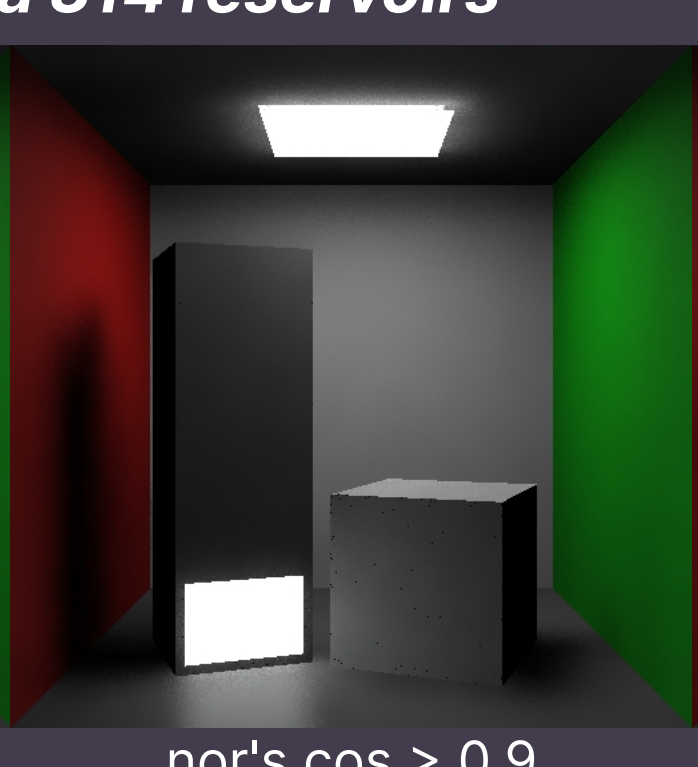
Radius 10 - Area 314 reservoirs



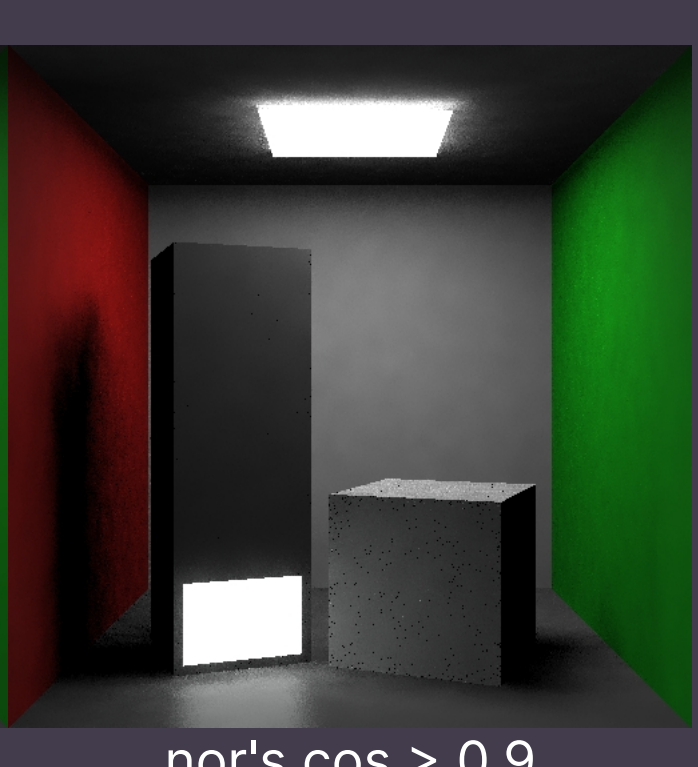
Start point
no restriction on spatial
average join: 13583.87
100%



nor's cos > 0.9
pos's dist < 0.5
average join: 12783.13
47.62%

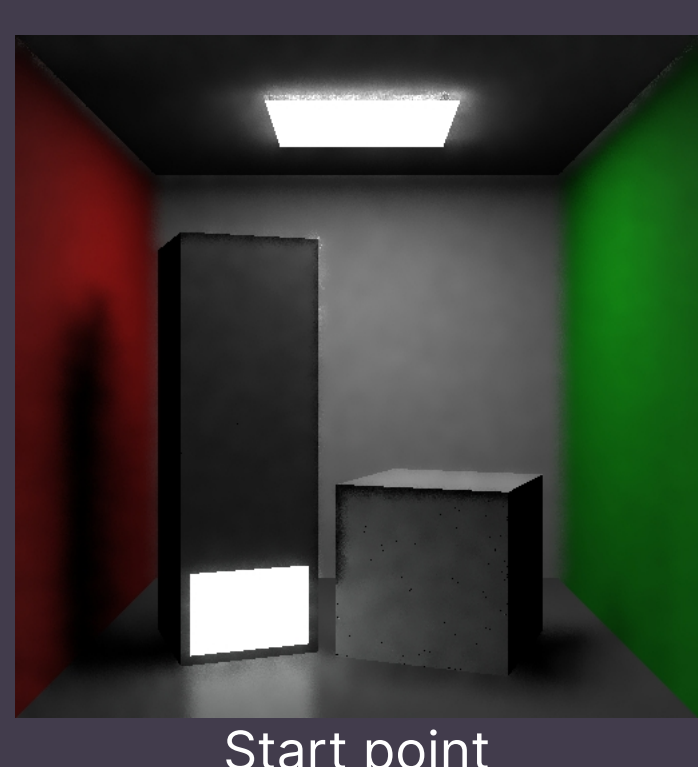


nor's cos > 0.9
pos's dist < 0.1
average join: 6833.63
27.17%

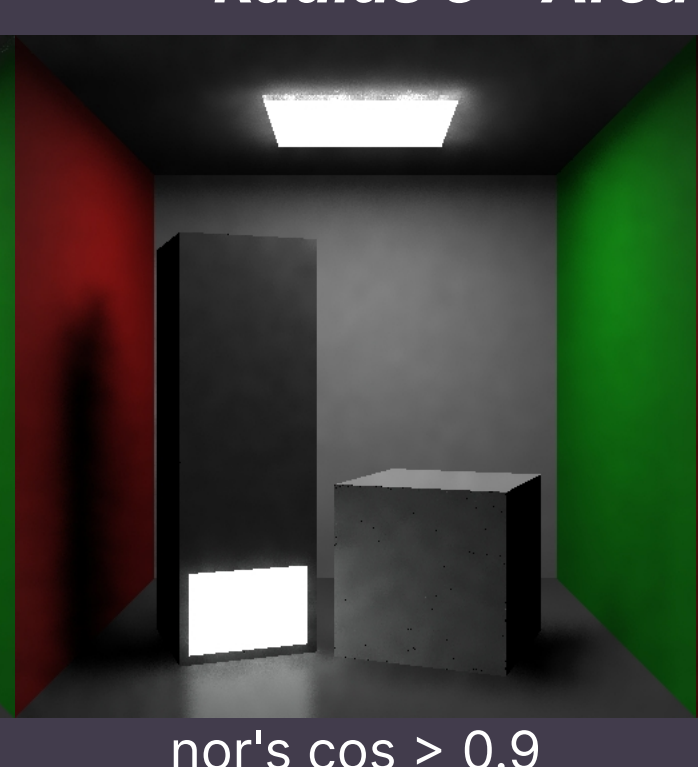


nor's cos > 0.9
pos's dist < 0.04
average join: 334.35
1.40%

Radius 5 - Area 78.5 reservoirs



Start point
no restriction on spatial
average join: 14521.27
100%

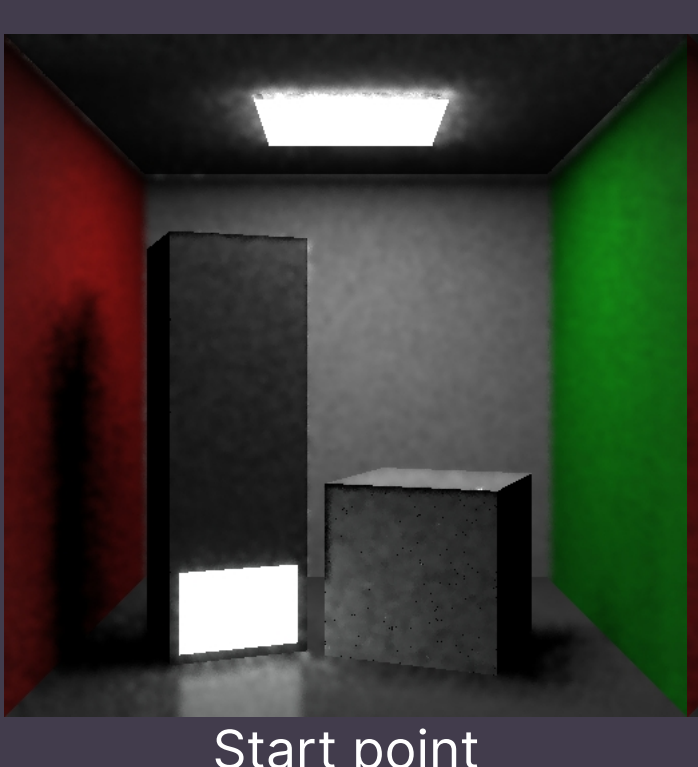


nor's cos > 0.9
pos's dist < 0.5
average join: 14092.63
97.04%

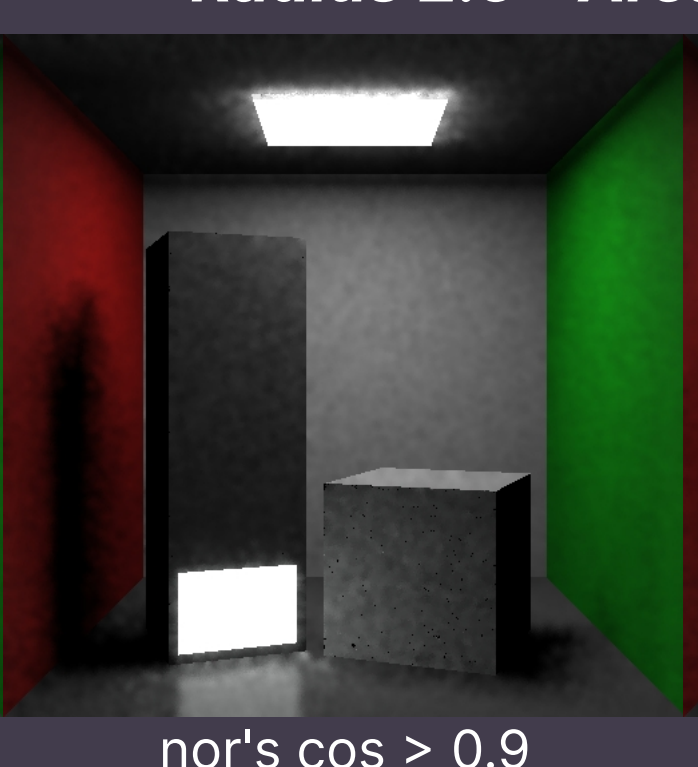
nor's cos > 0.9
pos's dist < 0.1
average join: 13095.53
90.18%

nor's cos > 0.9
pos's dist < 0.04
average join: 5553.23
38.24%

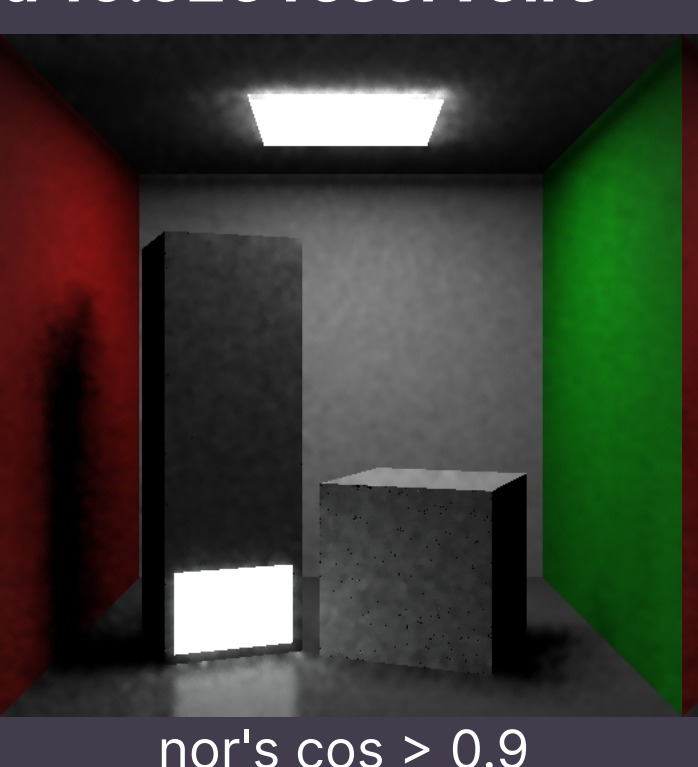
Radius 2.5 - Area 19.625 reservoirs



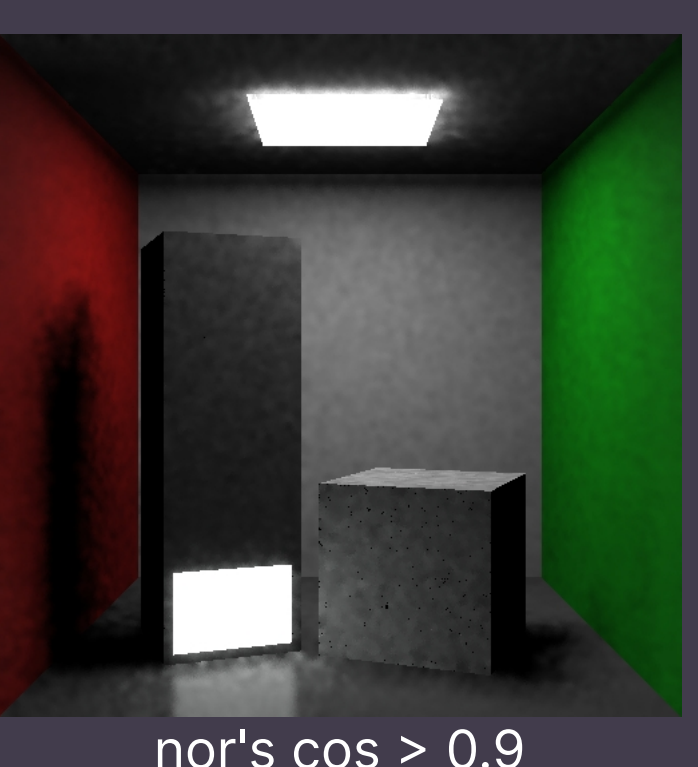
Start point
no restriction on spatial
average join: 15011.42
100%



nor's cos > 0.9
pos's dist < 0.5
average join: 14743.55
98.21%



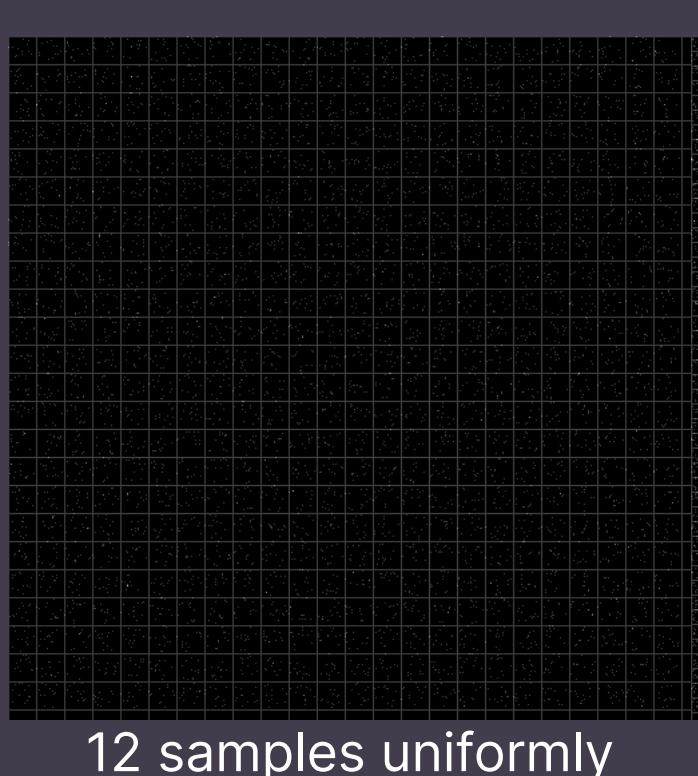
nor's cos > 0.9
pos's dist < 0.1
average join: 14698.51
97.91%



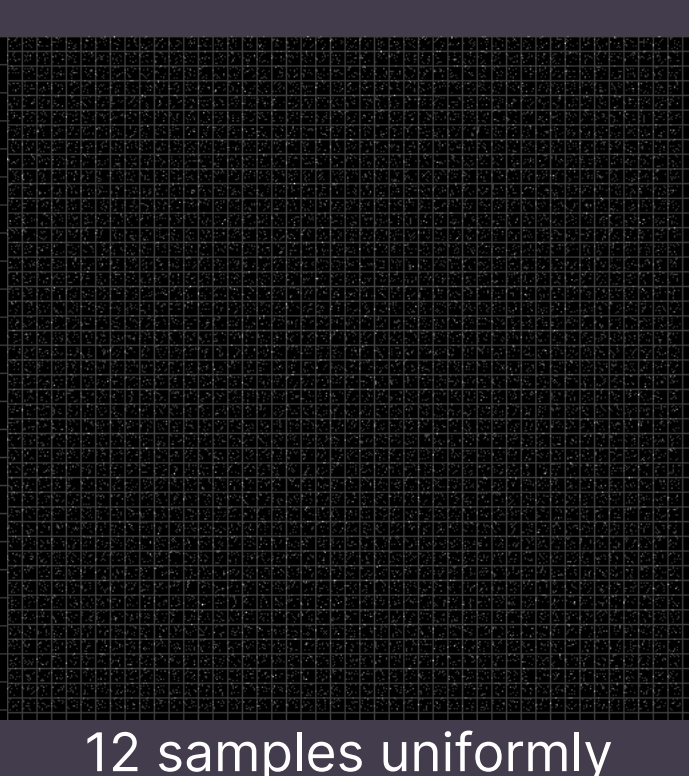
nor's cos > 0.9
pos's dist < 0.04
average join: 11608.62
77.33%

Ignoring GGX half's cos and gradually concentrating area results in better joining percentage, better specular and better shadows but it gradually adds fireflies. Oh, boy wouldn't it be nice if we can join shadow separately?

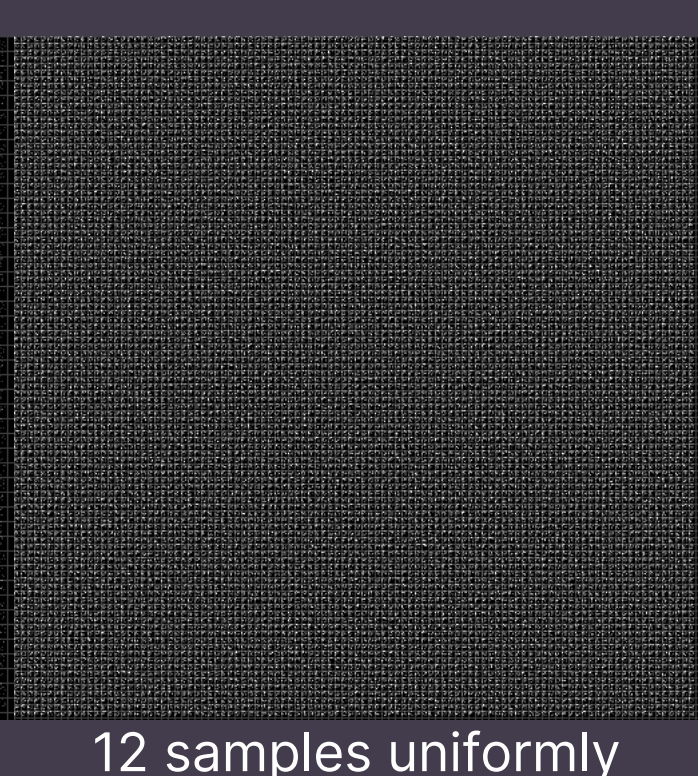
Lambert + GGX, 5 x MIS(Area Lights Importance, GGX), Temporal 4, Spatial 12 samples small disk area



12 samples uniformly distributed over radius 10
Occasionally collision.

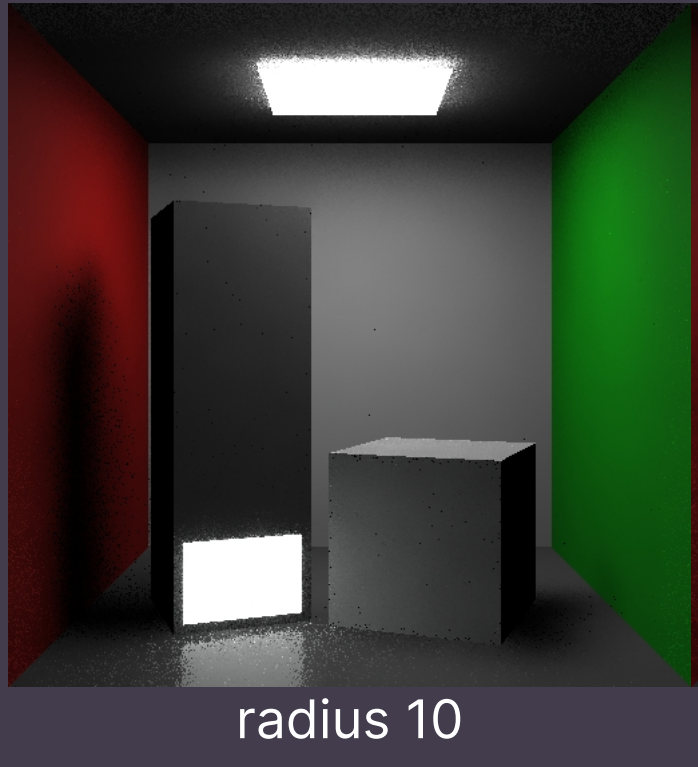


12 samples uniformly distributed over radius 5
There's collision.



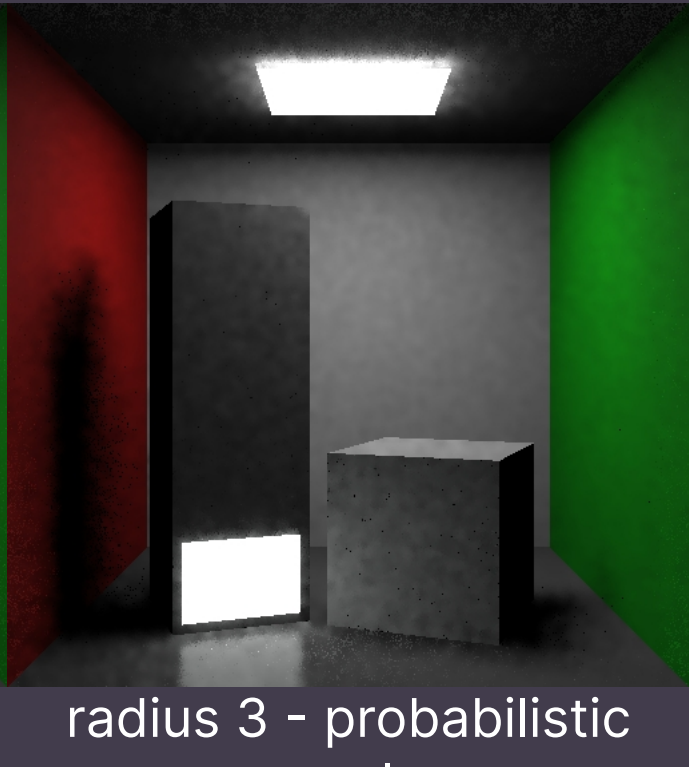
12 samples uniformly distributed over radius 2.5
Collision city.

For small disk areas instead of computing the sample using r = √(rand), angle = rand * 2 * π we could accept disk samples based on the probability p=num_samples/disk_area_samples. On average 12 disk samples will be generated without collision. This implementation, while it offered good specular for half's cos > 0.9 still has fireflies.

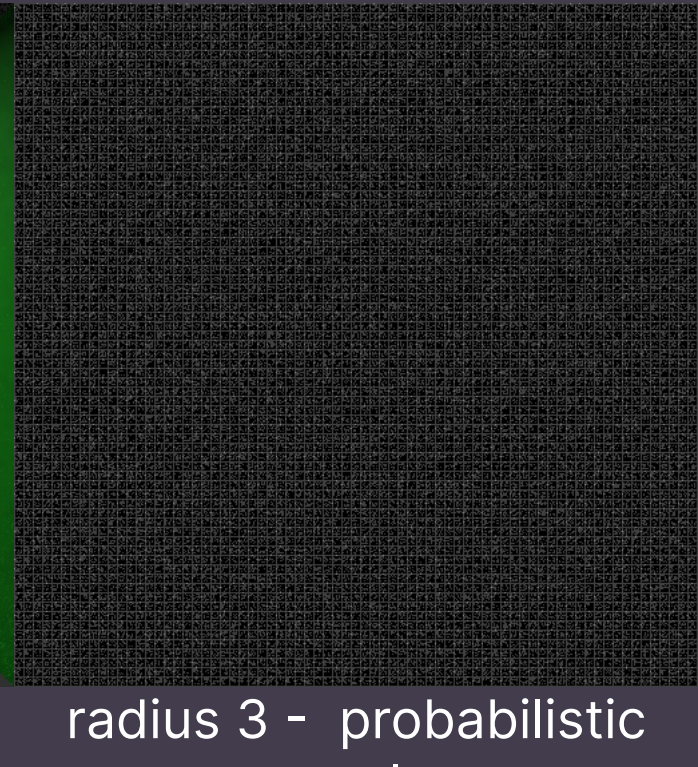


radius 10

half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.1
average join: 3692.08
27.17%



radius 3 - probabilistic samples
half's cos >= 0.9
nor's cos > 0.9
pos's dist < 0.1
average join: 11420.60
67.68%

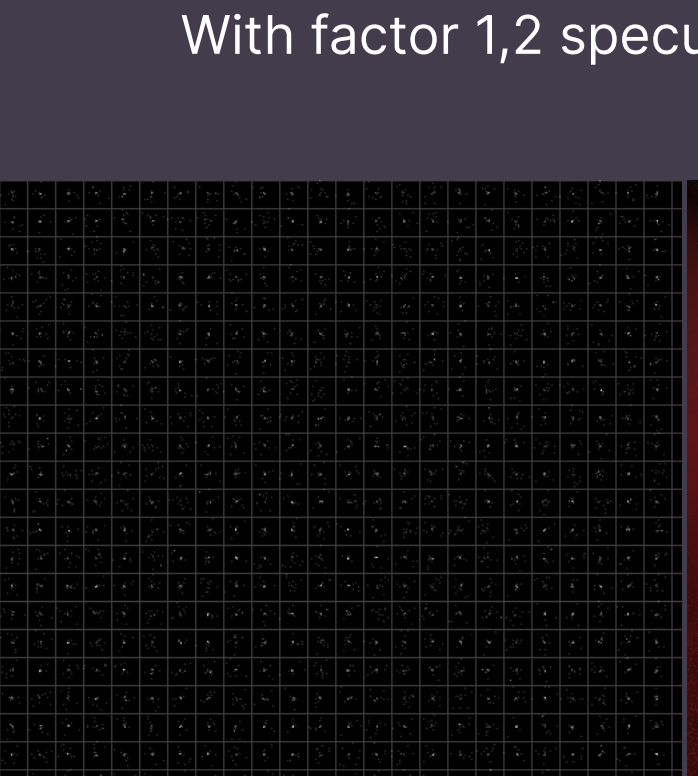


radius 3 - probabilistic samples

No collision.
Fireflies.

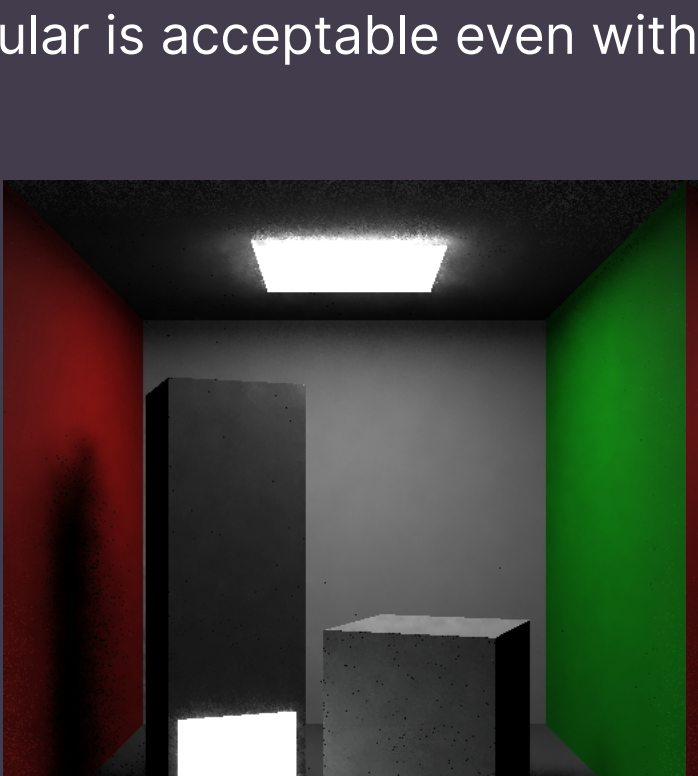
In the disk sampler instead of using r = √(rand) we could use r = rand ** factor and a factor of 1 or 2 would bias samples closer to the center. Changing the distribution requires integration of pdf in reservoir joining for unbiased results.

With factor 1,2 specular is acceptable even with half's cos >= 0.9, shadows are improved and fireflies have been reduced.



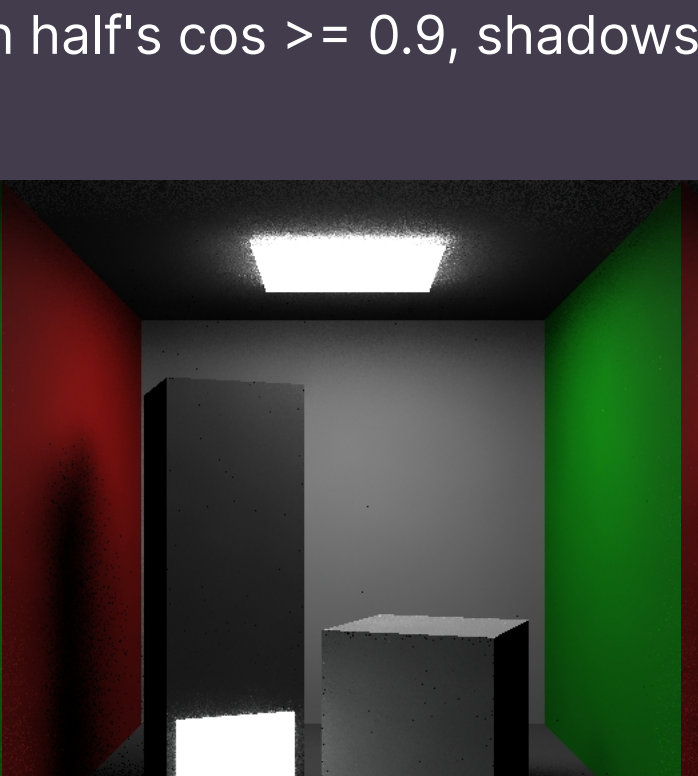
radius 10 factor 2

Collisions.
Reduced fireflies

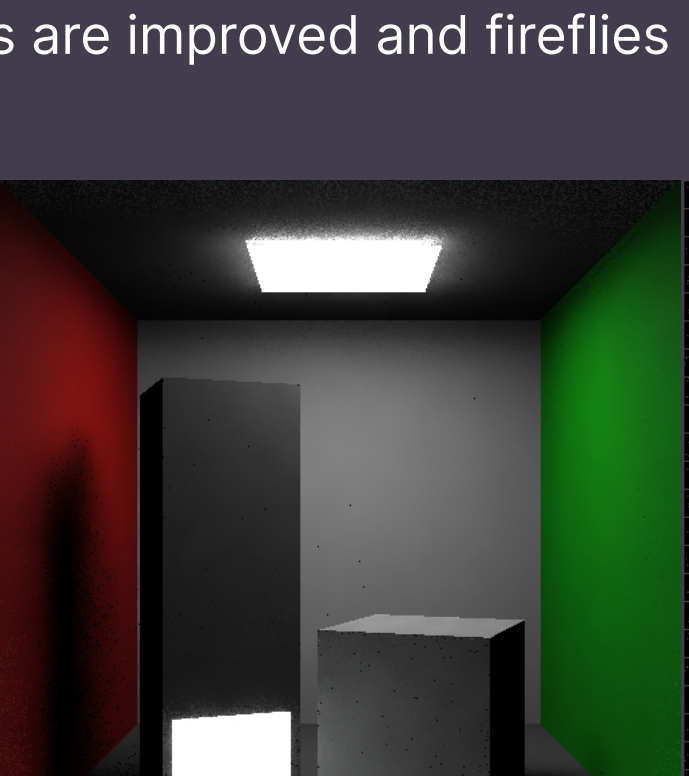


radius 10 factor 2

half's cos >= 0.9
nor's cos >= 0.9
pos's dist < 0.1
average join: 7526.37
52.38%

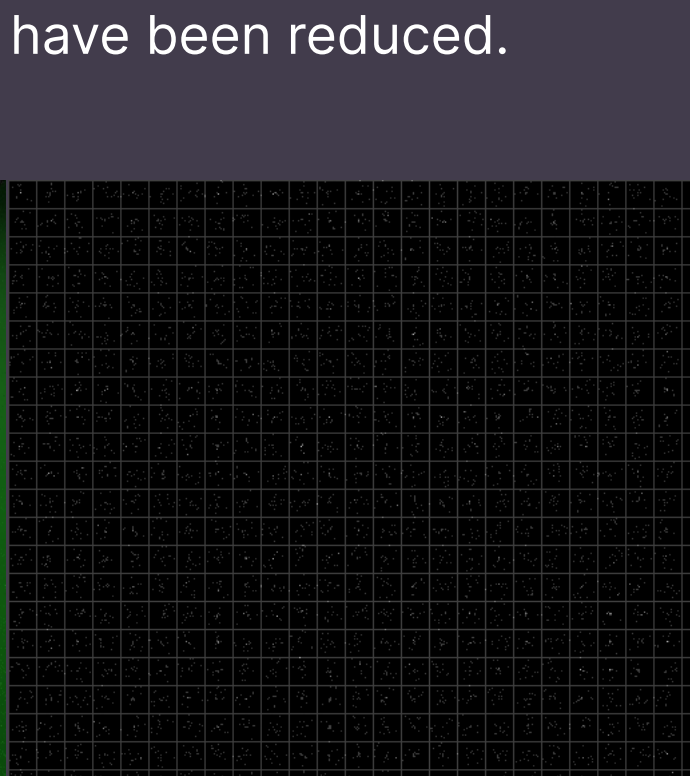


radius 10 factor 1/2
[uniform reference]
half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.1
average join: 3692.08
27.17%



radius 10 factor 1

half's cos >= 0.9
nor's cos > 0.9
pos's dist < 0.1
average join: 6049.12
42.38%

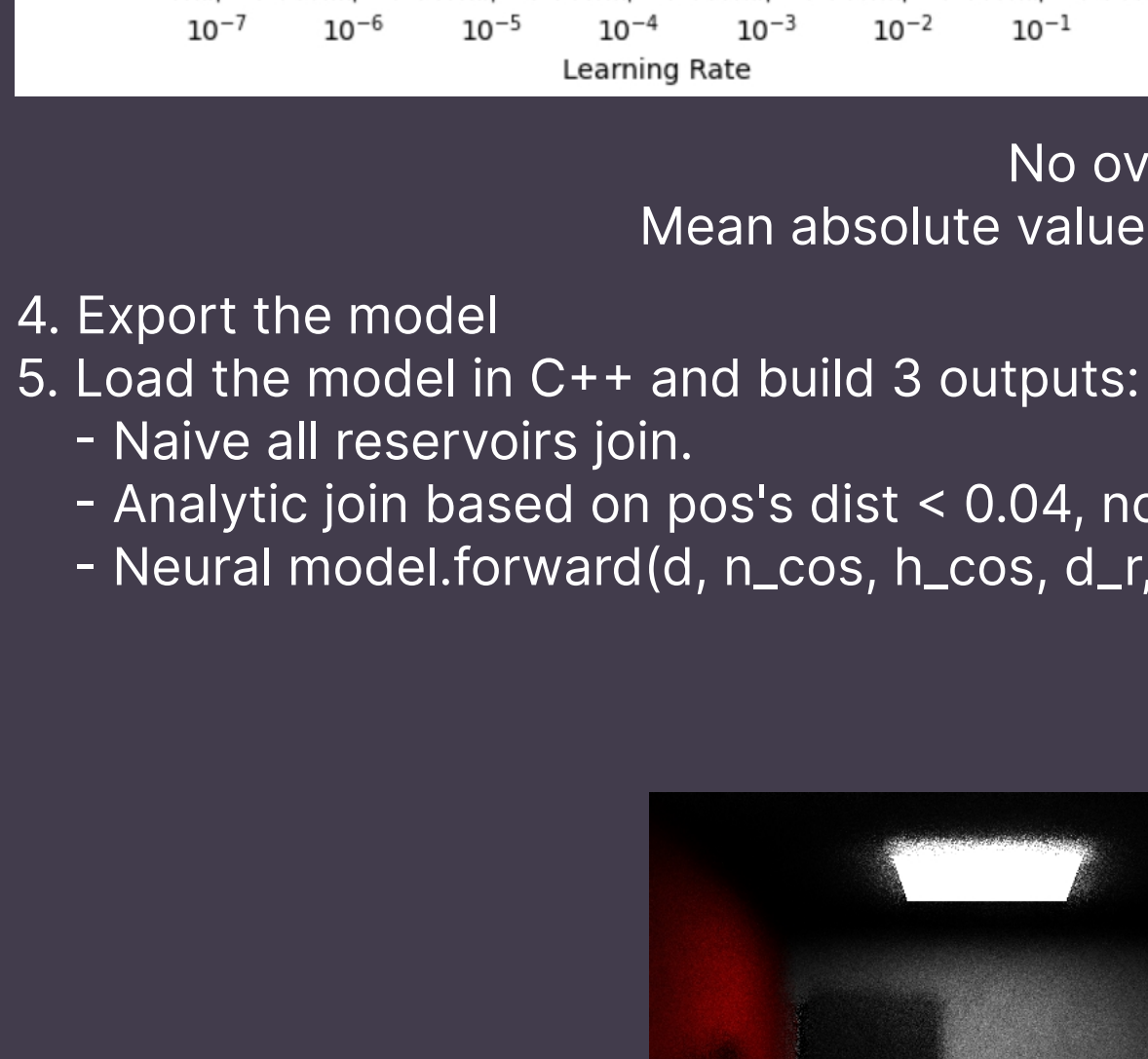


radius 10 factor 1

Less collisions.
Even less fireflies.

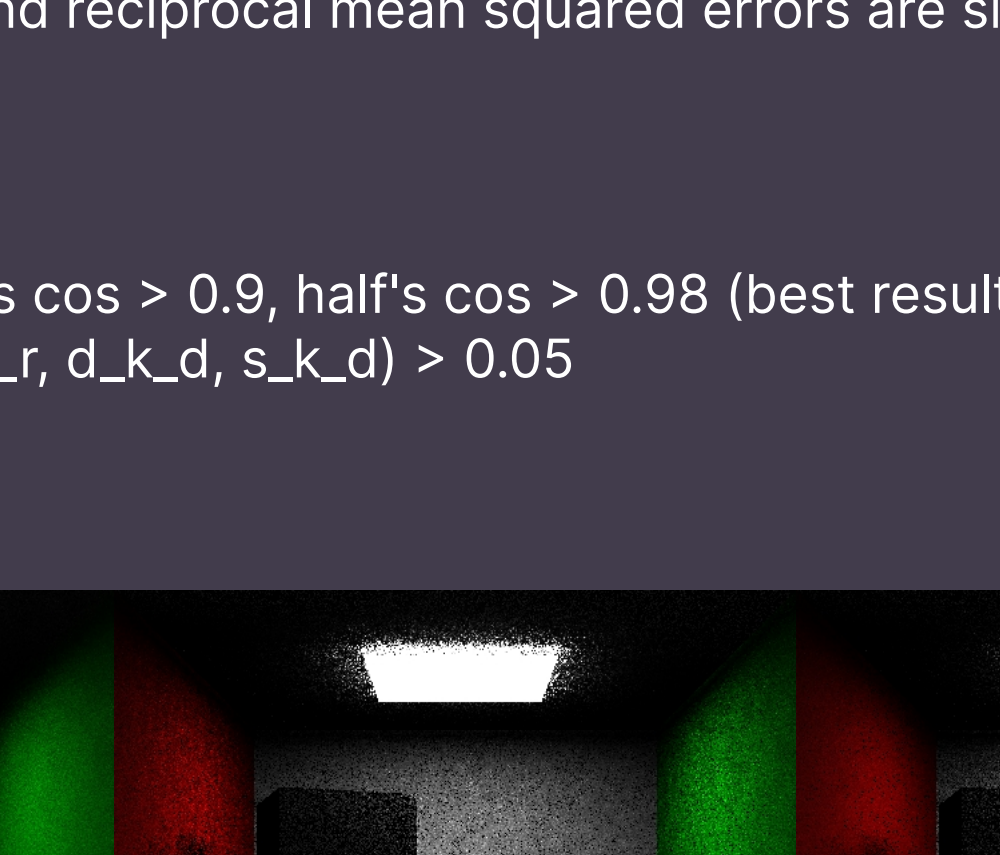
Neural based joining of 10 spatial samples from radius 10

Method:
1. Init reservoirs from 64 MIS (Area, GGX) + shadow pass and output the scene gbuffer & reservoirs into json's.
2. Generate 2.1M training samples in C++ as following:
   - record all joins from spatial (2.1M joins)
   - for each join, compute the loss with and without it against a reference image (scene rendered with Blender Cycles).
   - per sample record:
     - d ← distance between joining reservoirs (normalized in respect to scene).
     - n_cos ← cosine between normals.
     - h_cos ← cosine between half vectors.
     - d_r ← destination reservoir roughness.
     - s_r ← source reservoir roughness.
     - d_k_d ← destination reservoir albedo.
     - s_k_d ← source reservoir albedo.
     - delta_loss = (loss_without_join - loss_with_join).
   - normalize delta_loss sample.delta_loss /= max(|min_delta_loss|, max_delta_loss) min, max delta loss computed for all samples.
3. Train 2-hidden-layer Multilayer Perceptron (MLP) with 64 neurons per hidden layer and ReLU activations. Data was broken into 3 parts: 80% training data (in batches of 4096), 5% validation and 5% final test.



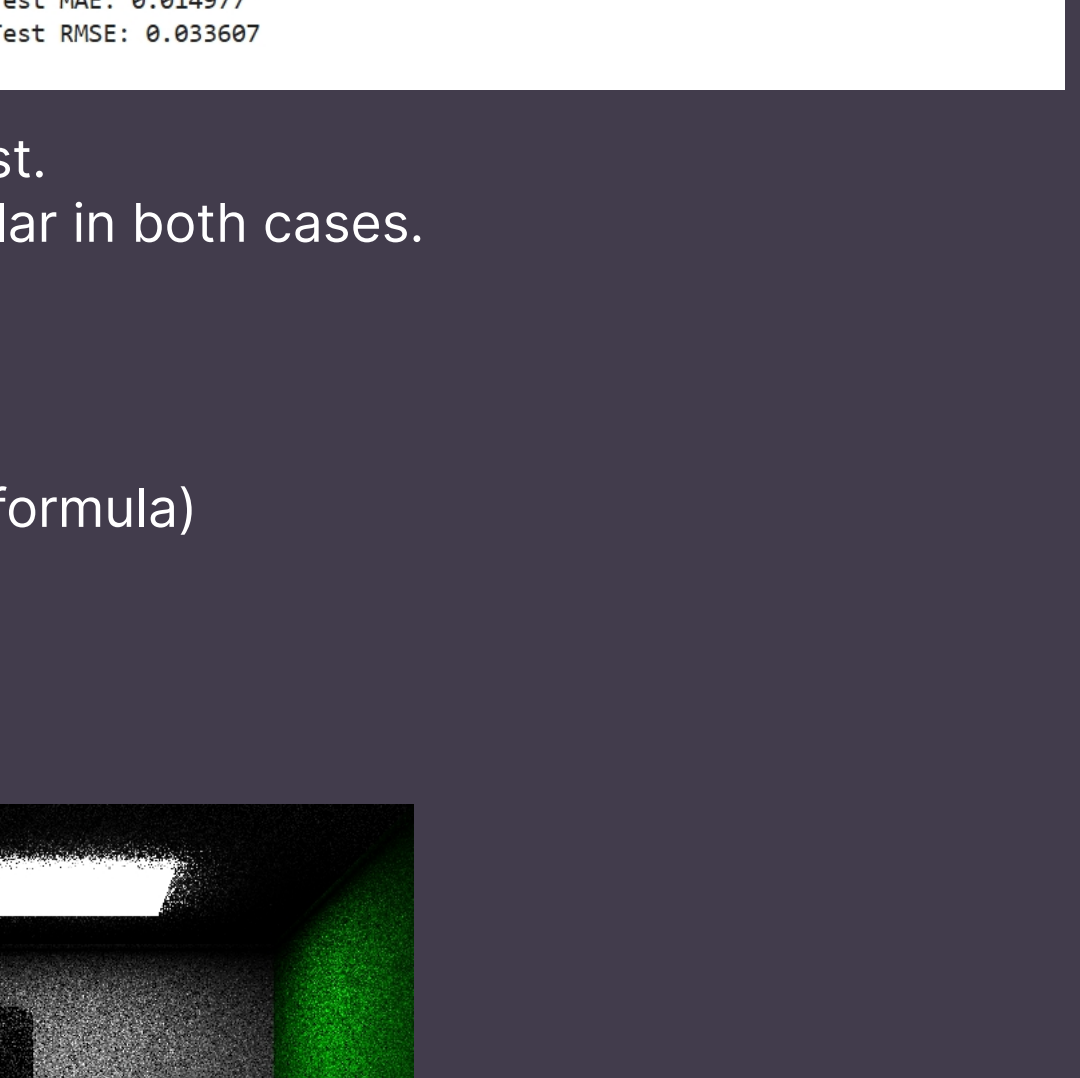
```
learn.lr_find()
```

```
SuggestedLRs(valley=0.0004786300996784121)
```



```
learn.fit_one_cycle(10, lr_max=0.00047863)
```

epoch	train_loss	valid_loss	mae	custom_rmse	time
0	0.001198	0.001153	0.015408	0.033952	00:25
1	0.001156	0.001144	0.015106	0.033829	00:25
2	0.001156	0.001156	0.016437	0.034003	00:26
3	0.001143	0.001141	0.015349	0.033780	00:26
4	0.001137	0.001136	0.015005	0.033711	00:26
5	0.001136	0.001139	0.015728	0.033748	00:26
6	0.001149	0.001132	0.015019	0.033650	00:25
7	0.001136	0.001132	0.014962	0.033644	00:26
8	0.001144	0.001132	0.015062	0.033642	00:26
9	0.001144	0.001131	0.014924	0.033630	00:26



```
learn.model.eval()
```

```
Model(
  (net): Sequential(
    (0): Linear(in_features=11, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=1, bias=True)
  )
)
```

```
X_test_device = X_test.to(device)
y_test_device = y_test.to(device)
```

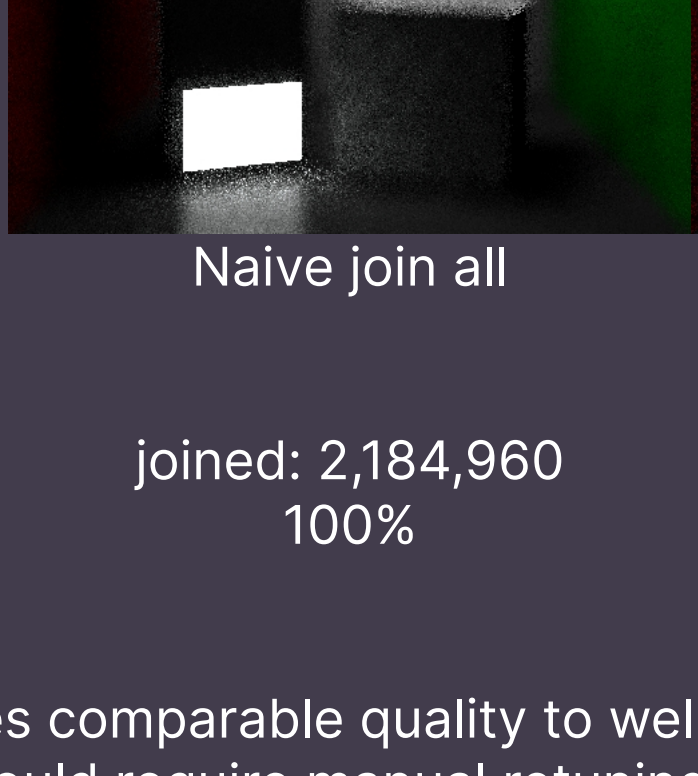
```
with torch.no_grad():
    predictions = learn.model(X_test_device)
    test_loss = nn.MSELoss()(predictions, y_test_device)
    test_mae = torch.abs(predictions - y_test_device).mean()
    test_rmse = torch.sqrt(((predictions - y_test_device) ** 2).mean())
```

```
print(f"Test Loss (MSE): {test_loss:.6f}")
print(f"Test MAE: {test_mae:.6f}")
print(f"Test RMSE: {test_rmse:.6f}")
```

```
Test Loss (MSE): 0.001129
Test MAE: 0.014977
Test RMSE: 0.033607
```

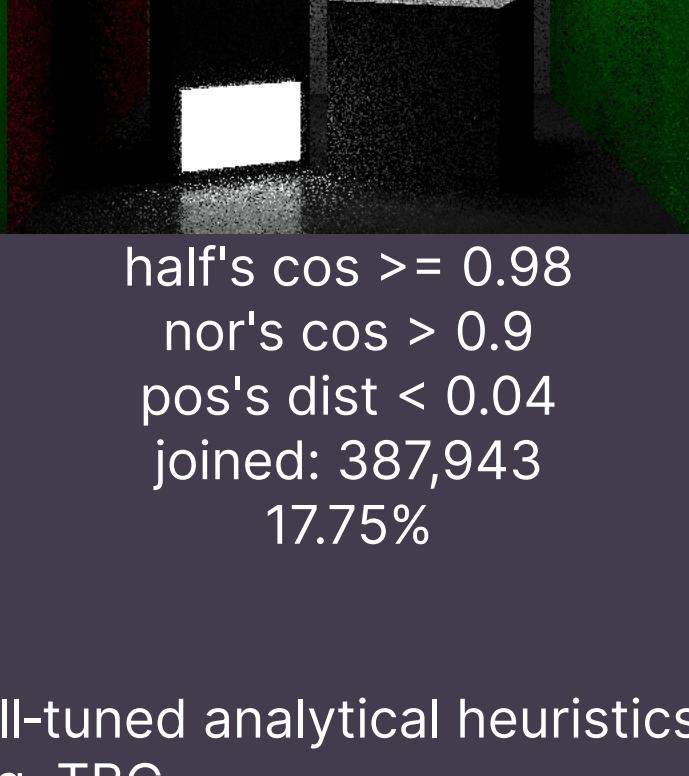
No overfit was recoreded in validation and final test.
Mean absolute value and reciprocal mean squared errors are similar in both cases.

4. Export the model
5. Load the model in C++ and build 3 outputs:
   - Naive all reservoirs join.
   - Analytic join based on pos's dist < 0.04, nor's cos > 0.9, half's cos > 0.98 (best results formula)
   - Neural model.forward(d, n_cos, h_cos, d_r, s_r, d_k_d, s_k_d) > 0.05

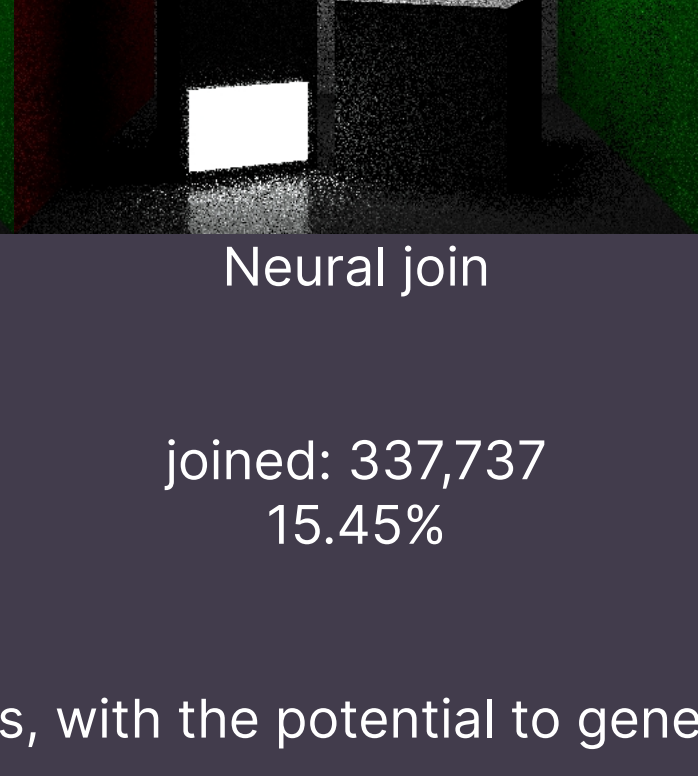


Naive join all

joined: 2,184,960
100%



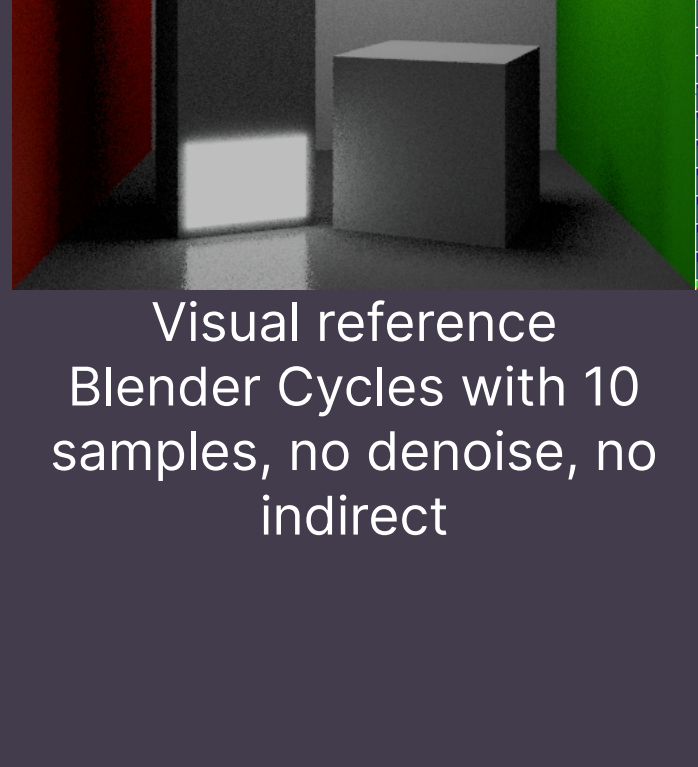
half's cos >= 0.98
nor's cos > 0.9
pos's dist < 0.04
joined: 387,943
17.75%



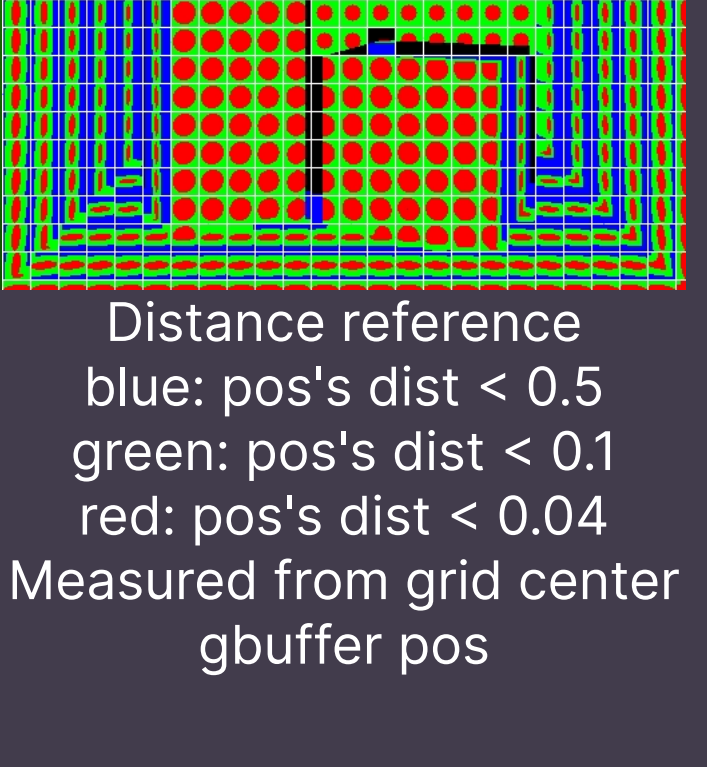
Neural join

joined: 337,737
15.45%

The neural model achieves comparable quality to well-tuned analytical heuristics, with the potential to generalise across scenes where fixed thresholds would require manual retuning. TBC.



Visual reference
Blender Cycles with 10 samples, no denoise, no indirect



Distance reference
blue: pos's dist < 0.5
green: pos's dist < 0.1
red: pos's dist < 0.04
Measured from grid center gbuffer pos

Reducing the sampling radius/count based on depth wasn't considered due to small scene depth range.